Phân tích 3 Cổng của LSTM (Phần 1)





1. Cổng Quên (Forget Gate - f(t))

- ✓ Nhận vào **h**(t-1) và **x**(t), tính toán bằng hàm Sigmoid.
- ✓ Quyết định phần nào của Trạng thái dài hạn **c**(t-1) sẽ bị 'bỏ đi'.

Ví dụ: Thấy chủ ngữ mới thì quên chủ ngữ cũ.



2. Cổng Cập nhật & Ghi (Input Gate - i(t), g(t))

g(t): Tạo ra vector giá trị ứng viên mới (dùng tanh).

i(t): Quyết định (bằng hàm Sigmoid) mức độ quan trọng của các ứng viên này để cộng vào **c**(t) làm thông tin mới.

Gemini Notebook

Phân tích 3 Cổng của LSTM (Phần 2)

Cập nhật Trạng thái Tế bào (c(t))

$$c(t) = f(t) \otimes c(t-1) + i(t) \otimes g(t)$$

(Quên đi quá khứ không cần thiết + Nhớ thêm thông tin mới)



3. Cổng Xuất (Output Gate - o(t))

Quyết định phần nào của Trạng thái Tế bào **c(t)** sẽ được đọc ra và đưa vào Trạng thái ẩn **h(t)**.

$$o(t) = \sigma(W_o[x(t), h(t-1)])$$

$$h(t) = o(t) \otimes \tanh(c(t))$$

Gemini Notebook

Sử dụng LSTM trong Keras

Cực kỳ đơn giản, chỉ cần thay SimpleRNN bằng LSTM. Mọi công thức cổng phức tạp bên trên đã được Keras tối ưu bằng thư viện CUDNN dưới nền.

```
model = keras.Sequential([
    keras.layers.LSTM(20, return_sequences=True, input_shape=[None, 1]),
    keras.layers.LSTM(20),
    keras.layers.Dense(1)
])
```



Hiệu năng: LSTM giải quyết tốt chuỗi dài khoả hàng trăm bước thời gian, vượt xa SimpleRNN (chỉ hiệu quả với < 20 bước).

Gemini Notebook

Tế bào GRU (Gated Recurrent Unit)

GRU (2014) là một nỗ lực đơn giản hóa cấu trúc của LSTM.

Difference 1



1. Gộp Trạng thái:

GRU không có đường ray c(t) riêng biệt. Nó gộp chung Trạng thái ẩn h(t) và Trạng thái tế bào làm một.

Difference 2



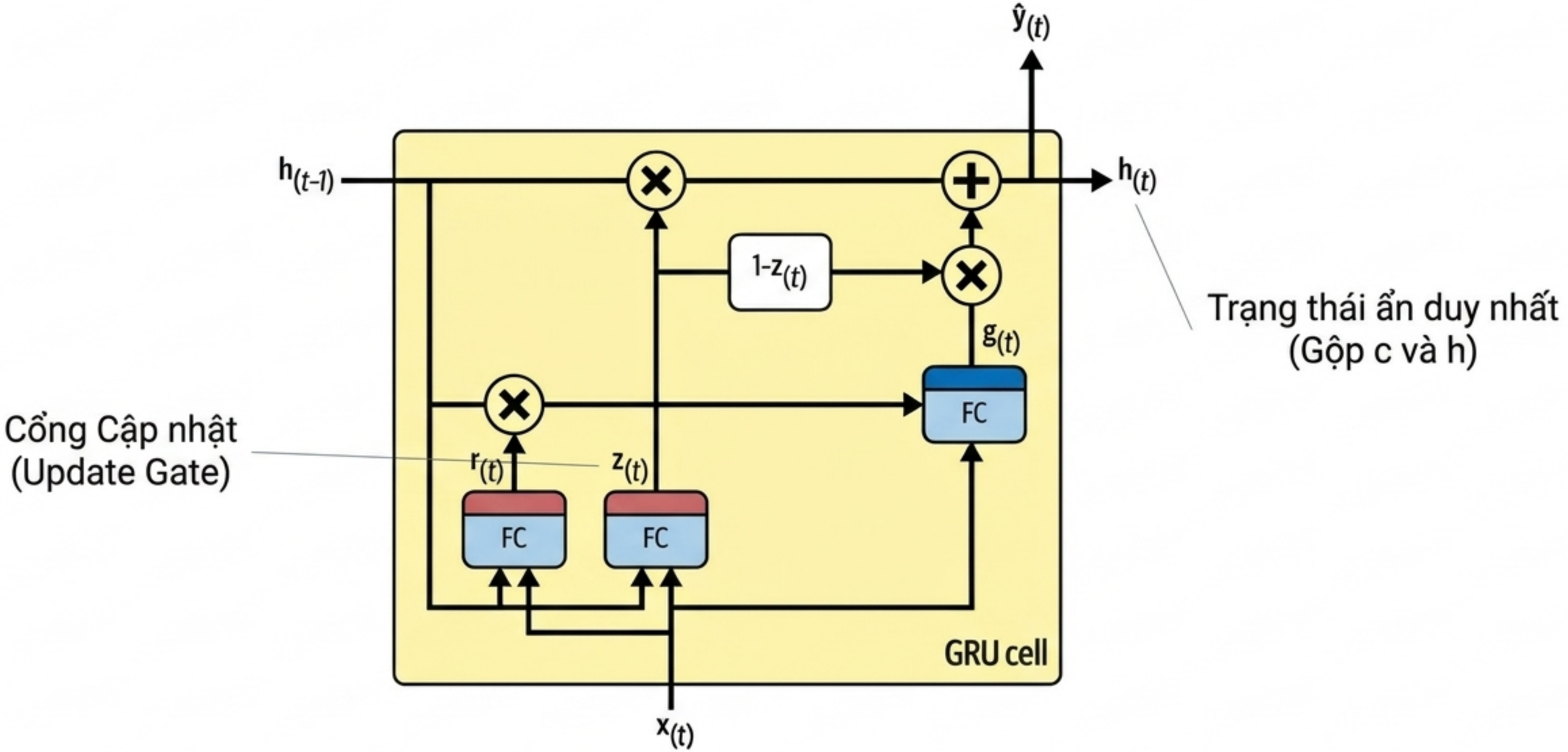
2. Gộp Cổng:

Gộp Cổng Quên và Cổng Cập nhật thành một Cổng Cập nhật (Update Gate - z(t)).

- Nếu z = 1: Lưu mới
- Nếu z = 0: Giữ cũ

Gemini Notebook

Cấu tạo Tế bào GRU



Hình 15-13: Cấu tạo Tế bào GRU

Gemini Notebook

So sánh LSTM và GRU

Đặc điểm	LSTM	GRU
Năm ra đời	1997	2014
Trạng thái duy trì	h_t và c_t	Chỉ h_t
Số lượng Cổng	3 (Quên, Nhập, Xuất)	2 (Cập nhật, Reset)
Tốc độ huấn luyện	Chậm 	Nhanh hơn 
Số lượng tham số	Lớn (Nặng RAM)	Thấp hơn (~ 25% ít hơn)
Độ chính xác	Tốt với chuỗi rất phức tạp	Ngang ngửa LSTM ở đa số Data



Lời khuyên thực tiễn: Bắt đầu thử nghiệm với GRU vì nó nhanh. Nếu thấy hiệu năng có vẻ bị thắt nút, chuyển sang LSTM xem có cải thiện không.

Gemini Notebook

Từ Tuần tự sang Song song



Điểm yếu chết người của RNN (kể cả LSTM/GRU)

Tính toán tuần tự.

Bạn không thể tính bước thời gian t = 100 nếu chưa tính xong bước t = 99.

Hậu quả: Không thể chạy song song trên phần cứng GPU một cách hiệu quả.



Giải pháp: Mạng Tích chập (CNN 1D)

Kernel trượt độc lập với nhau -> Hoàn toàn song song hóa được.

Cách thức: Dùng CNN 1D (chỉ quét theo chiều dài thời gian) để trích xuất đặc trưng chuỗi.

Gemini Notebook

Vượt qua giới hạn bằng Tích chập Giãn nở

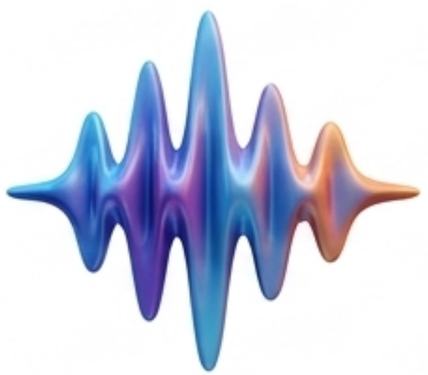


Vấn đề của CNN 1D thông thường

- Bộ lọc (Kernel) có kích thước **nhỏ** (VD: size 3) -> Mỗi nơ-ron lớp 1 chỉ nhìn thấy 3 bước thời gian.
- Để nhìn được 100 bước thời gian (Trường thụ cảm - Receptive field lớn) -> Phải **xếp chồng hàng chục lớp** CNN 1D + Pooling.
- Kết quả: Rất nặng nề, mất thông tin chuỗi chi tiết. Cần cơ chế quét khác!



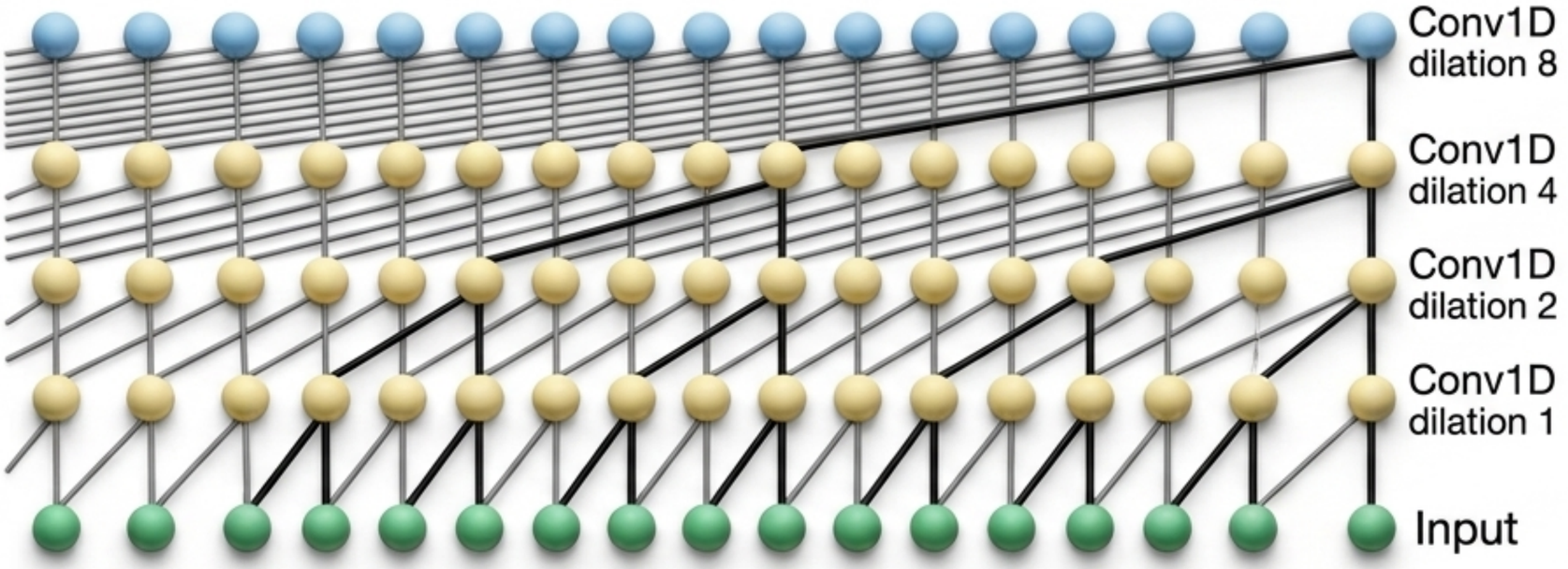
Giải pháp: WaveNet của DeepMind



Chuyên tạo **giọng nói Audio** (chuỗi cực kỳ dài, tới 16,000 mẫu/giây). Thay vì CNN thường, nó sử dụng **Tích chập giãn nở** (**Dilated Convolutions**).

Gemini Notebook

Tích chập Giãn nở (Dilated Convolutions) trong WaveNet



Hình 15-14: *Kiến trúc WaveNet (Dilated Conv)*

Tỷ lệ giãn (Dilation rate) tăng dần theo lũy thừa 2^n (1, 2, 4, 8...):

- **Lớp 1 (rate=1):** Kernel trượt từng ô liền kề (Giống hệt Conv1D thường).
- **Lớp 2 (rate=2):** Kernel bỏ qua 1 ô xen kẽ -> 1 nơ-ron bao phủ được khoảng cách 4 ô.
- **Lớp 3 (rate=4):** Kernel bỏ qua 3 ô xen kẽ -> 1 nơ-ron nhìn được 8 ô.

Nhờ tăng rate theo lũy thừa, mạng chỉ cần **10 lớp** là đã **có Trường thụ cảm** bao phủ được **1024 bước thời gian** mà số lượng **tham số vẫn cực kỳ nhỏ**.

Gemini Notebook

Code: Xây dựng WaveNet với Keras

Ta dùng vòng lặp để tạo các lớp Conv1D giãn nở:

```
model = keras.Sequential()
model.add(keras.layers.InputLayer(input_shape=[None, 1]))

# Cac lop tích chap gian nò (1, 2, 4, 8) lạp lại
for rate in (1, 2, 4, 8) * 2:
    model.add(keras.layers.Conv1D(
        filters=20, kernel_size=2, padding="causal",
        activation="relu", dilation_rate=rate))

model.add(keras.layers.Conv1D(filters=10, kernel_size=1))
model.add(keras.layers.Dense(1)) # Lop du doan
```



Lưu ý quan trọng: Tham số padding='causal' cực kỳ quan trọng ở Time Series. Nó chèn 0 vào bên trái để Kernel không bao giờ với tới thông tin của 'tương lai'.

Gemini Notebook

Tổng kết Kiến thức (Phần 1 - RNN Cơ bản)



Sequence Data

Đòi hỏi mô hình có bộ nhớ để theo dõi thứ tự của dữ liệu.



RNN Cơ bản

- Chia sẻ trọng số qua các bước thời gian.
- Dùng Trạng thái ẩn h(t) để mang thông tin tới tương lai.



Ứng dụng RNN đa dạng

- Seq2Seq: Dự báo, Dịch máy
- Seq2Vec: Phân tích cảm xúc
- Encoder-Decoder: Chatbot

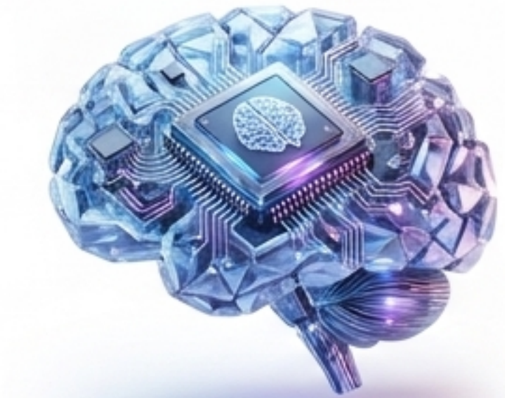


BPTT & Tiền xử lý

- BPTT: Mở cuộn mạng qua không gian thời gian (dễ gây Gradient bốc hơi hoặc bùng nổ khi chuỗi quá dài).
- Tiền xử lý (Differencing): Rất hữu ích để loại bỏ tính thời vụ và xu hướng trước khi đưa vào mạng.

Gemini Notebook

Tổng kết Kiến thức (Phần 2 - Các Kiến trúc Tiên tiến)



Trí nhớ Dài Hạn (LSTM & GRU)

Cơ chế: Thiết kế Tế bào thông minh với các "Cổng" (Gate) động đóng mở bằng hàm Sigmoid.

Giá trị cốt lõi: Giúp bảo toàn Gradient đi xa, duy trì thông tin qua nhiều bước thời gian một cách hiệu quả, mang lại "**Trí nhớ Dài Hạn**".



Bứt phá Tốc độ (CNN 1D & WaveNet)

Cơ chế: Một hướng đi song song (không tuần tự), xử lý nhanh hơn RNN rất nhiều.

Giá trị cốt lõi: Sử dụng Tích chập giãn nở (Dilated Convolutions) để nắm bắt và bao phủ chuỗi độ dài hàng chục ngàn bước một cách nhẹ nhàng (Ví dụ: Ứng dụng Âm thanh).

Gemini Notebook